

Viva Questions & Answers

Experiment 1 - HTML Department Page (Viva Q&A)

Q1. What is the difference between `<b>` and `<strong>` tags?

Ans: `<b>` only makes text visually bold, while `<strong>` also gives the text semantic importance, which is useful for screen readers and SEO.

Q2. What is the difference between `<i>` and `<em>`?

Ans: `<i>` italicizes text purely for style, whereas `<em>` italicizes it to indicate emphasis, carrying semantic meaning.

Q3. What does the `target="_blank"` attribute do in an anchor tag?

Ans: It opens the linked page in a new browser tab or window instead of the current one.

Q4. Which attributes are used to control the size of an image in HTML?

Ans: The width and height attributes (in pixels or other units) control the displayed size of the `<img>` element.

Q5. What are `<table>`, `<tr>`, `<th>`, and `<td>` used for?

Ans: `<table>` defines a table, `<tr>` defines a row, `<th>` defines a bold/centered header cell, and `<td>` defines a normal data cell.

Q6. What is the difference between `<ul>` and `<ol>`?

Ans: `<ul>` creates an unordered (bulleted) list while `<ol>` creates an ordered (numbered) list.

Q7. What is the purpose of the `<iframe>` tag?

Ans: It embeds another HTML document (such as a webpage) inside the current page, displayed in its own rectangular frame.

Q8. What is the difference between the old `<frame>/<frameset>` tags and `<iframe>`?

Ans: `<frame>/<frameset>` split the whole browser window into multiple documents and are now obsolete in HTML5, while `<iframe>` embeds a single external document inline within the normal page layout.

Experiment 2 - JavaScript Form Validation (Viva Q&A)

Q1. What is form validation and why is it needed?

Ans: Form validation checks that user input meets required rules (length, format, non-empty) before it is processed, preventing incorrect or incomplete data from being submitted.

Q2. What is the difference between client-side and server-side validation?

Ans: Client-side validation runs in the browser using JavaScript for instant feedback, while server-side validation runs on the server to enforce security, since client-side checks can be bypassed.

Q3. How is an empty field checked in JavaScript?

Ans: By comparing the field's value to an empty string, e.g. `if(fieldValue == "") { ... }`.

Q4. How does the code check for a valid email format?

Ans: It checks that the email string contains both an '@' symbol and a '.' using `indexOf()`, returning -1 if either is missing.

Q5. What does the `isNaN()` function do?

Ans: It checks whether a value is 'Not-a-Number' and returns true if the value cannot be converted to a valid number, which is used here to confirm the phone number is numeric.

Q6. What is the purpose of the `onsubmit` event handler on a form?

Ans: It runs a JavaScript function when the form is submitted, and if that function returns false, the form submission is cancelled.

Q7. How do you prevent a form from submitting when validation fails?

Ans: By returning false from the validation function called in `onsubmit`, which stops the default form submission behavior.

Q8. Why is a minimum length check applied to the password field?

Ans: To enforce a basic security requirement, ensuring passwords are not too short and easy to guess.

Experiment 3 - React Class Components, Events, Lifecycle (Viva Q&A)

Q1. What is the difference between a class component and a functional component in React?

Ans: A class component is defined using an ES6 class extending `React.Component` and uses lifecycle methods and `this.state`, while a functional component is a plain JavaScript function that uses Hooks like `useState` for state management.

Q2. What is the `useState` hook used for?

Ans: It lets a functional component hold and update local state, returning the current state value and a function to update it.

Q3. What is a synthetic event in React?

Ans: It is React's cross-browser wrapper around the browser's native event, ensuring events behave consistently across different browsers.

Q4. How is a click event handled in React?

Ans: By passing a function reference to the `onClick` prop of an element, e.g. `<button onClick={handleClick}>`.

Q5. What is the React component lifecycle?

Ans: It is the sequence of phases a class component goes through: mounting, updating, and unmounting, each with corresponding lifecycle methods.

Q6. What is the purpose of `componentDidMount()`?

Ans: It runs once immediately after the component is first rendered to the DOM, commonly used for initial data fetching or setup.

Q7. What is the purpose of `componentDidUpdate()`?

Ans: It runs after the component re-renders due to a state or prop change, useful for reacting to updates such as fetching new data.

Q8. What is the difference between state and props?

Ans: State is data managed within a component and can change over time, while props are read-only values passed into a component from its parent.

Experiment 4 - Order Details List (Viva Q&A)

Q1. How is the quantity of each item updated when a button is clicked?

Ans: Each item has its own state variable (e.g. q1, q2, q3) created with useState, and the button's onClick handler calls the corresponding setter function to increase it by 1.

Q2. How is the total cost calculated dynamically?

Ans: The total is computed by multiplying each item's quantity by its price and summing the results, recalculated automatically on every render since it depends on the state values.

Q3. Why does the UI update automatically when quantity changes?

Ans: Because calling a state setter function (like setQ1) triggers React to re-render the component with the new state value.

Q4. Which JavaScript object is used to get the current date, and how?

Ans: The built-in Date object; new Date() returns the current date and time.

Q5. How is the expected delivery date calculated as current date + 3 days?

Ans: By using delivery.setDate(order.getDate() + 3), which adds 3 days to the order date's day value.

Q6. What does toString() do?

Ans: It converts a Date object into a readable string containing just the date portion (day, month, year), without the time.

Q7. Why is a separate state variable used for each product's quantity instead of one shared variable?

Ans: So each product's quantity can be tracked and updated independently without affecting the others.

Experiment 5 - React Registration Form (Viva Q&A)

Q1. What is a controlled component in React?

Ans: An input element whose value is controlled by React state, updated via an onChange handler, so the component's state is always the single source of truth for the input's value.

Q2. What is the role of the onChange event handler in the form?

Ans: It updates the corresponding state variable (like fname, lname, email) every time the user types into the input field.

Q3. How does the form decide which message to display?

Ans: The register() function checks each field in order and calls setMsg() with an appropriate error message if a field is empty, or a success message if all fields are filled.

Q4. What is conditional rendering in React?

Ans: Displaying different UI content based on a condition, such as showing an error or success message depending on the validation result.

Q5. Why is a single msg state variable used instead of separate error states for each field?

Ans: It keeps the example simple by showing one message at a time; a more advanced form could use separate error states per field for simultaneous multi-field validation.

Q6. What triggers the register() function to run?

Ans: The onClick event of the Register button.

Experiment 6 - Node.js & Express REST API (Viva Q&A)

Q1. What is a REST API?

Ans: An architectural style for building web services where resources are accessed and manipulated using standard HTTP methods like GET, POST, PUT, and DELETE.

Q2. What is Express.js?

Ans: A minimal and flexible Node.js web framework that simplifies building web servers and APIs by providing routing, middleware, and request/response handling.

Q3. What is the purpose of app.use(express.json())?

Ans: It is middleware that parses incoming requests with JSON payloads, making the data available on req.body.

Q4. What HTTP method is used to retrieve data, and which is used to create new data?

Ans: GET is used to retrieve data, while POST is used to create new data.

Q5. What is the difference between PUT and DELETE in this API?

Ans: PUT updates an existing resource identified by its id, while DELETE removes a resource identified by its id.

Q6. What is the difference between req.params and req.body?

Ans: req.params holds values from the URL path (like an id in /books/:id), while req.body holds data sent in the request body, typically for POST or PUT requests.

Q7. What does app.listen(3000, ...) do?

Ans: It starts the Express server and makes it listen for incoming HTTP requests on port 3000.

Q8. What is middleware in Express?

Ans: A function that has access to the request and response objects and runs during the request-response cycle, often used for parsing, logging, or authentication.

Experiment 7 - Node.js CRUD with MongoDB (Viva Q&A)

Q1. What is Mongoose?

Ans: An Object Data Modeling (ODM) library for MongoDB and Node.js that provides schema-based modeling and simplifies database interactions.

Q2. What is a Schema in Mongoose?

Ans: A blueprint that defines the structure, field types, and validation rules for documents in a MongoDB collection.

Q3. What is a Model in Mongoose?

Ans: A constructor compiled from a Schema that provides an interface for creating, reading, updating, and deleting documents in the corresponding MongoDB collection.

Q4. What does Student.insertMany() do?

Ans: It inserts multiple documents into the Student collection in a single operation.

Q5. What is the difference between updateOne() and updateMany()?

Ans: updateOne() updates only the first document matching the filter, while updateMany() updates all documents that match the filter.

Q6. Why is async/await used in the performCRUD() function?

Ans: Because Mongoose database operations are asynchronous and return Promises; async/await lets the code wait for each operation to complete before moving to the next, keeping the logic readable.

Q7. What does mongoose.connect() do?

Ans: It establishes a connection between the Node.js application and the specified MongoDB database using a connection string.

Q8. Why is mongoose.connection.close() called at the end?

Ans: To properly close the database connection once all CRUD operations are complete, freeing up resources.